

系统开发工具基础

第一周实验报告

作者：魏尚儒

学号：25020007136

链接：

<https://github.com/1583e4d-Rn/2026summer-reports>

2026 年 8 月 31 日

目录

1	实验内容	2
1.1	含空格文件名的批量整理	2
1.2	随机访问日志统计	4
1.3	制造、解决并解释一次合并冲突	7
1.4	修复并构建一页技术说明	9
2	课后练习	12
2.1	Shell	12
2.1.1	挂载硬盘分区	12
2.1.2	符号 > 与 » 的区别	12
2.1.3	Shell 语法中的部分符号	13
2.1.4	chmod 的符号模式	14
2.1.5	管道与 grep 配合实现快速查找	15
2.2	Git 相关	16
2.2.1	使用 SSH 密钥简化推送步骤	16
2.2.2	向远程仓库推送代码	17
2.3	Latex	18
2.3.1	文档结构控制	18
2.3.2	创建无序列表与有序列表	19
2.3.3	插入图片	20
3	感悟	21

1 实验内容

1.1 含空格文件名的批量整理

本题要求如下：

- 创建 input/docs 和 input/tmp; 创建 input/docs/notes one.txt (内容为 alpha 和 beta 两行)、input/docs/.secret.txt (内容为 hidden)、input/tmp/empty.txt (空文件) 以及 input/run.log (任意两行日志)。
- 显示 q01 的绝对路径, 并以长格式列出 input 下全部项目, 包含隐藏文件。
- 把 input 下所有.txt 文件复制到 work/< 学号 >/, 保留相对目录结构并正确处理名称中的空格。
- 将复制后的目录权限设为 750、普通文件权限设为 640; 生成 inventory.txt, 列出相对路径和字节数。

递归创建目录结构

使用 `mkdir -p` 一次性建立 q01/input/docs 和 q01/input/tmp。

```
mkdir -p q01/input/{docs,tmp}
```

创建含内容文件与空文件

```
echo -e "alpha\nbeta" > q01/input/docs/notesone.txt
echo "hidden" > q01/input/docs/secret.txt
touch q01/input/tmp/empty.txt
echo -e "log1\nlog2" > q01/input/run.log
```

显示绝对路径与长格式列表

```
cd q01  
pwd  
ls -la input/
```

安全复制所有.txt 文件（处理空格）

```
STUDENT_ID=25020007136  
mkdir -p work/$STUDENT_ID  
find . -name "*.txt" -type f -exec cp --parents {} "../work/$STUDENT_ID/" \;
```

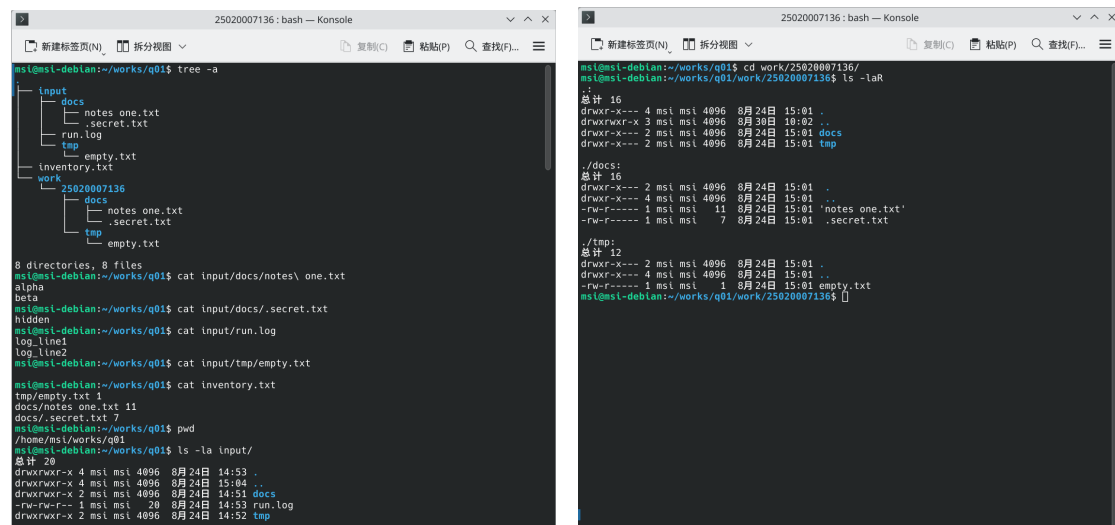
批量设置目录和文件权限

```
cd work/$STUDENT_ID  
find . -type d -exec chmod 750 {} \;  
find . -type f -exec chmod 640 {} \;
```

生成清单 inventory.txt（相对路径 + 字节数）

```
cd "work/$STUDENT_ID"  
find . -name "*.txt" -type f -printf "%P %s\n" > ../../inventory.txt
```

运行效果如下：



1.2 随机访问日志统计

编写 analyze.sh 脚本完成日志统计

本题要求编写一个 Shell 脚本 analyze.sh，功能如下：

- 接受一个 CSV 文件路径作为唯一参数，文件不存在时输出错误到 stderr 并返回非零退出码。
- 输出 HTTP 5xx 数量最多的前 2 个 path（按次数降序，若相同按字典序）。
- 计算全部数据行的平均 latency_ms（表头不计），保留两位小数。
- 不得使用 Python、电子表格或手工统计。

脚本如下：

```
#!/usr/bin/env bash

# 检查参数个数是否为 1
```

```
if [ $# -ne 1 ]; then
    echo "Usage: $0 <csv-file>" >&2
    exit 1
fi

csv_file="$1"

# 检查文件是否存在
if [ ! -f "$csv_file" ]; then
    echo "Error: File '$csv_file' not found" >&2
    exit 1
fi

echo "Top 2 5xx paths:"

grep -v '^timestamp' "$csv_file" \ # 去掉表头行

    # 提取第 4 列 (status) 为 5xx 的, 输出第 3 列 (path)
    | awk -F, '$4 >= 500 && $4 < 600 {print $3}' \

    | sort \ # 排序以便 uniq 计数
    | uniq -c \ # 统计每个 path 出现次数
    | sort -k1,1nr -k2,2 \ # 先按次数降序, 次数相同按 path 字典序
    | head -2 \ # 取前两名
    | awk '{print $2, $1}' # 将输出格式调整为 "path 次数"

# 计算平均延迟, 跳过表头并保留两位小数
avg=$(awk -F, 'NR > 1 {sum += $5; count++}
END {if (count) printf "%.2f", sum/count; else print "0"}' "$csv_file")
```

```
echo "Average latency: $avg ms"
```

运行脚本

使用题目给定的 access.csv 运行：

```
chmod +x analyze.sh  
./analyze.sh access.csv
```

输出如下：

```
Top 2 5xx paths:  
/api/orders 3  
/api/users 2  
Average latency: 193.75 ms
```

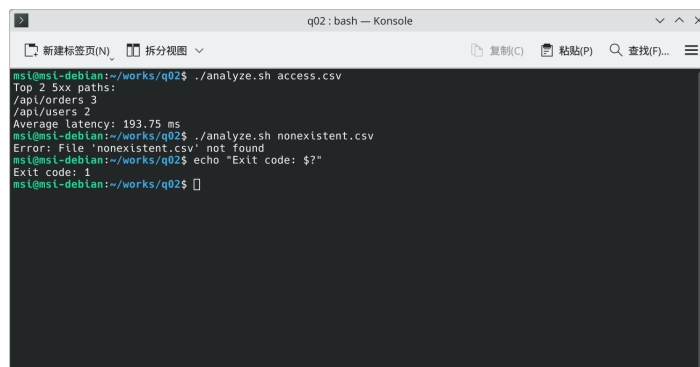
对不存在的文件运行：

```
./analyze.sh nonexistent.csv  
echo "Exit code: $?"
```

输出 (stderr)：

```
Error: File 'nonexistent.csv' not found  
Exit code: 1
```

运行效果如下：



```
msl@msl-debian:~/works/q02$ ./analyze.sh access.csv
Top 2 5xx paths:
/apl/orders 3
/apl/users 2
Average latency: 193.75 ms
msl@msl-debian:~/works/q02$ ./analyze.sh nonexistent.csv
Error: File 'nonexistent.csv' not found
msl@msl-debian:~/works/q02$ echo "Exit code: $?"
Exit code: 1
msl@msl-debian:~/works/q02$
```

1.3 制造、解决并解释一次合并冲突

本题要求主动制造、解决并解释一次合并冲突，详细如下：

- 初始化仓库，在 main 创建 config.txt，内容为 mode=normal，并完成初始提交。
- 创建 feature-a，把内容改为 mode=safe 并提交；从初始 main 创建 feature-b，把内容改为 mode=fast 并提交。
- 回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 mode=safe，并另加一行 note=reviewed。
- 确认工作区干净，运行 git log -all -graph -decorate -oneline 查看历史。

初始化 Git 仓库并提交

首次提交前，需先设置姓名与邮件。

```
mkdir -p q03 && cd q03
git init
echo "mode == normal" > config.txt
git add config.txt
git commit -m "Initial commit with normal mode declaration"
```

签出两个分支并各自修改

```
git checkout -b feature-a
echo "mode == safe" > config.txt
git add config.txt
git commit -m "Set feature a to safe mode"

git checkout main
git checkout -b feature-b
echo "mode == fast" > config.txt
git add config.txt
git commit -m "Set feature b to fast mode"
```

合并分支并解决冲突

```
git checkout main
git merge feature-a    # 无冲突
git merge feature-b    # 产生冲突
# 手动编辑 config.txt, 保留 "mode == safe" 并新增 "note == reviewed"
git add config.txt
git commit -m "Conflict solved"
```

检查工作区并查看历史

```
git status
git log --all --graph --decorate --oneline
```

显示工作区干净，并输出提交历史。全部的输出如下：

```
msl@msl-debian:~/works/q03$ git init
已初始化空的 Git 仓库于 /home/msl/works/q03/.git/
msl@msl-debian:~/works/q03$ git branch -m main
msl@msl-debian:~/works/q03$ echo "mode=normal" > "config.txt"
msl@msl-debian:~/works/q03$ git add config.txt
msl@msl-debian:~/works/q03$ git commit -m "Initial commit with normal mode"
作者身份未知

*** 请告诉我你是谁。

运行

git config --global user.email "you@example.com"
git config --global user.name "Your Name"

来设置您账号的缺省身份标识。
如果仅在本仓库设置身份标识, 则省略 --global 参数。

致命错误: 无法自动探测邮件地址 (得到 'msl@msl-debian (none)')
msl@msl-debian:~/works/q03$ git config user.email "mail@example.com"
msl@msl-debian:~/works/q03$ git config user.name "Name F"
msl@msl-debian:~/works/q03$ git commit -m "Initial commit with normal mode declaration"
[main (根提交) b7f5c0f] Initial commit with normal mode declaration
1 file changed, 1 insertion(+)
create mode 100644 config.txt
msl@msl-debian:~/works/q03$ git checkout -b feature-a
切换到新分支 'feature-a'
msl@msl-debian:~/works/q03$ echo "modesafe" > "config.txt"
msl@msl-debian:~/works/q03$ git add config.txt
msl@msl-debian:~/works/q03$ git commit -m "Set feature a to safe mode"
[feature-a 3088557] Set feature a to safe mode
1 file changed, 1 insertion(+)
msl@msl-debian:~/works/q03$ git checkout main
切换到分支 'main'
msl@msl-debian:~/works/q03$ git checkout -b feature-b
切换到新分支 'feature-b'
msl@msl-debian:~/works/q03$ echo "modefast" > "config.txt"
msl@msl-debian:~/works/q03$ git add config.txt
msl@msl-debian:~/works/q03$ git commit -m "Set feature b to fast mode"
[feature-b e91b5cb] Set feature b to fast mode
1 file changed, 1 insertion(+), 1 deletion(-)
msl@msl-debian:~/works/q03$ git checkout main
切换到分支 'main'
```

```
msl@msl-debian:~/works/q03$ git merge feature-a
更新 b7f5c0f..3088557
Fast-forward
 config.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
msl@msl-debian:~/works/q03$ git merge feature-b
自动合并 config.txt
冲突 (内容): 合并冲突于 config.txt
自动合并失败, 修正冲突然后提交修正的结果。
msl@msl-debian:~/works/q03$ vim config.txt
msl@msl-debian:~/works/q03$ git add config.txt
msl@msl-debian:~/works/q03$ git commit -m "Conflict solved"
[main 60d6f82] Conflict solved
msl@msl-debian:~/works/q03$ git status
位于分支 main
无文件要提交, 干净的工作区
msl@msl-debian:~/works/q03$ git log --all --graph --decorate --oneline
* 60d6f82 (HEAD -> main) Conflict solved
|
* e91b5cb (feature-b) Set feature b to fast mode
* | 3088557 (feature-a) Set feature a to safe mode
|/
* b7f5c0f Initial commit with normal mode declaration
msl@msl-debian:~/works/q03$
```

1.4 修复并构建一页技术说明

给定一段 latex 模板:

```
\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{姓名—学号}
\maketitle
\section{Result}
% 在此加入公式、表格及正文引用
\end{document}
```

要求:

- 加入一个带编号和 label 的公式, 并在正文中使用 ref 或 eqref 引用。
- 加入一个 2 行 2 列表格, 设置 caption 和 label, 并在正文中引用该表。

- 使用 `latexmk -pdf -halt-on-error` 或 `tectonic` 生成 `report.pdf`。
- 确认 PDF 能打开，且交叉引用不显示 “??”。

修改后代码

```
\documentclass{article}
\usepackage{amsmath}
\usepackage[UTF8]{ctex}
\begin{document}
\title{Tool Report}
\author{魏尚儒 -25020007136}
\maketitle

\section{Result}

% 引用
公式 ~\eqref{eq:energy}
表 ~\ref{tab:sample}

% 公式（编号会自动生成，可用\tagf{}手动添加）
\begin{equation}
E = m c^2 \label{eq:energy}
\end{equation}

% 2*2 表格
\begin{table}[h]
\centering
\caption{示例数据表}\label{tab:sample}
\begin{tabular}{|c|c|}
\hline
```

```
项目 & 数值 \\
\hline
A & 1.23 \\
B & 4.56 \\
\hline
\end{tabular}
\end{table}

\end{document}
```

预览效果如下：

Tool Report

魏尚儒-25020007136

2026 年 8 月 30 日

1 Result

公式 (1) 表 1

$$E = mc^2$$

(1)

表 1: 示例数据表

项目	数值
A	1.23
B	4.56

2 课后练习

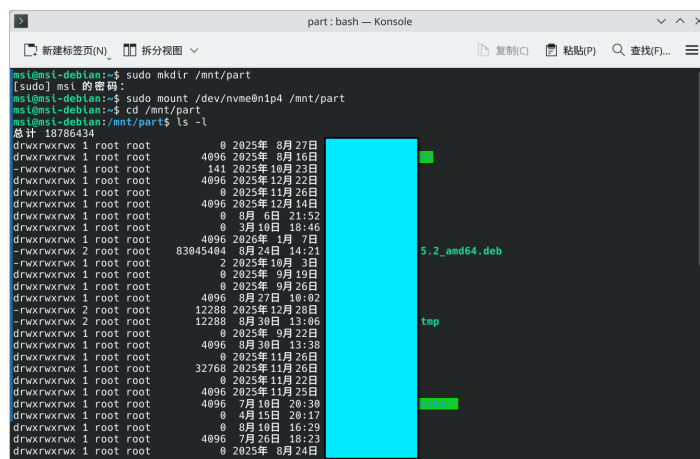
2.1 Shell

2.1.1 挂载硬盘分区

使用 `mkdir` 命令在 `/mnt` 下建立挂载点，再挂载指定的硬盘分区访问。

```
sudo mkdir /mnt/part
sudo mount /dev/nvme0n1p4 /mnt/part
cd /mnt/part
```

使用 `'ls -l'` 命令查看文件夹内容，可以看到成功挂载了此分区。

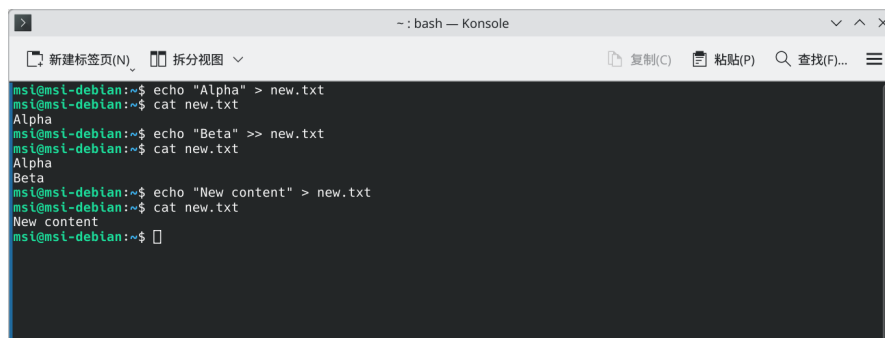


```
part: bash — Konsole
新建标签页(N) 拆分视图
复制(C) 粘贴(P) 查找(F)...
msl@msl-debian:~$ sudo mkdir /mnt/part
[sudo] msl 的密码:
msl@msl-debian:~$ sudo mount /dev/nvme0n1p4 /mnt/part
msl@msl-debian:~$ cd /mnt/part
msl@msl-debian:/mnt/part$ ls -l
总计 18766434
drwxrwxrwx 1 root root          0 2025年  8月 27日
drwxrwxrwx 1 root root       4096 2025年  8月 16日
-rwxrwxrwx 1 root root       141 2025年 10月 23日
drwxrwxrwx 1 root root       4096 2025年 12月 22日
drwxrwxrwx 1 root root          0 2025年 11月 26日
drwxrwxrwx 1 root root       4096 2025年 12月 14日
drwxrwxrwx 1 root root          0  8月  6日 21:52
drwxrwxrwx 1 root root          0  3月 10日 15:46
drwxrwxrwx 1 root root       4096 2026年  1月  7日
-rwxrwxrwx 2 root root    83045484 8月 24日 14:21  5.2_amd64.deb
-rwxrwxrwx 1 root root          2 2025年 10月  3日
drwxrwxrwx 1 root root          0 2025年  9月 19日
drwxrwxrwx 1 root root       4096 2025年  9月 26日
drwxrwxrwx 1 root root       4096  8月 27日 10:02
-rwxrwxrwx 2 root root    12288 2025年 12月 28日
-rwxrwxrwx 2 root root    12288  8月 30日 13:06  tmp
drwxrwxrwx 1 root root          0 2025年  9月 22日
drwxrwxrwx 1 root root       4096  8月 30日 13:38
drwxrwxrwx 1 root root          0 2025年 11月 26日
drwxrwxrwx 1 root root    32768 2025年 11月 26日
drwxrwxrwx 1 root root          0 2025年 11月 22日
drwxrwxrwx 1 root root       4096 2025年 11月 25日
drwxrwxrwx 1 root root       4096  7月 10日 20:30
drwxrwxrwx 1 root root          0  4月 15日 20:17
drwxrwxrwx 1 root root       4096  8月 10日 16:29
drwxrwxrwx 1 root root       4096  7月 26日 18:23
drwxrwxrwx 1 root root          0 2025年  8月 24日
```

2.1.2 符号 `>` 与 `>>` 的区别

在使用 `echo` 命令写入文件时，若原文件存在，使用符号 `>` 会覆盖文件中的内容，而符号 `>>` 会直接在文件末尾追加新内容。

下图显示了两符号的区别。（不指定 `-n` 时，`echo` 生成的文件每行末尾均带有换行符）



```
msi@msi-debian:~$ echo "Alpha" > new.txt
msi@msi-debian:~$ cat new.txt
Alpha
msi@msi-debian:~$ echo "Beta" >> new.txt
msi@msi-debian:~$ cat new.txt
Alpha
Beta
msi@msi-debian:~$ echo "New content" > new.txt
msi@msi-debian:~$ cat new.txt
New content
msi@msi-debian:~$
```

2.1.3 Shell 语法中的部分符号

与其他语言不同，Shell 语法（以 Bash 为例）的部分符号有其独特的用法。

给变量赋值时，等号两边不能有空格（以下是正确的变量赋值方式）

```
foo=bar
```

```
echo $foo
```

输出：*bar*

错误的赋值方式（会被解析为命令）

```
foo = bar
```

输出：*foo: command not found*

命令替换：将命令输出赋给变量

```
current_date=$(date)
```

```
echo "Today is $current_date"
```

双引号：变量会被替换

```
name="World"
```

```
echo "Hello, $name!"
```

输出：*Hello, World!*

```
# 单引号：变量不会被替换（原义字符串）
echo 'Hello, $name!'
# 输出：Hello, $name!

# 中括号需要与里面的内容用空格隔开
if [ ! -f "$csv_file" ]; then
    echo "Error: File '$csv_file' not found" >&2
    exit 1
fi
# 若无空格，系统会将两个连续的符号识别为一个操作符，进而报错或执行额外操作
```

2.1.4 chmod 的符号模式

除数字模式外，chmod 还可使用符号模式，相较数字模式更便于理解。

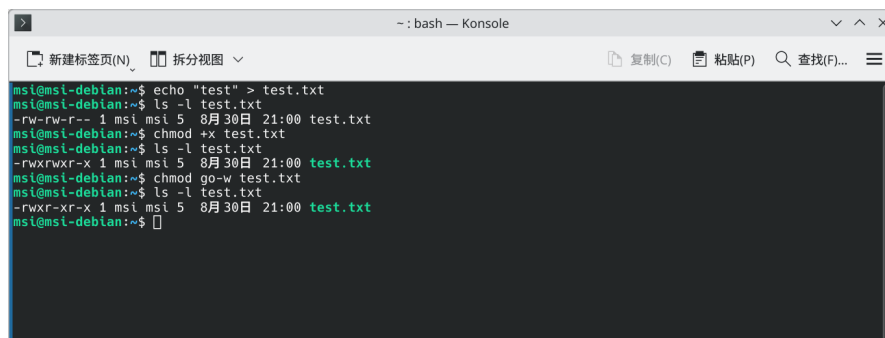
```
# 创建一个测试文件
echo "test" > test.txt

# 查看初始权限
ls -l test.txt

# 给所有用户添加执行权限
chmod +x test.txt
ls -l test.txt

# 移除组和其他用户的写权限
chmod go-w test.txt
ls -l test.txt
```

输出结果如下：



```
msi@msi-debian:~$ echo "test" > test.txt
msi@msi-debian:~$ ls -l test.txt
-rw-rw-r-- 1 msi msi 5 8月30日 21:00 test.txt
msi@msi-debian:~$ chmod +x test.txt
msi@msi-debian:~$ ls -l test.txt
-rwxrwxr-x 1 msi msi 5 8月30日 21:00 test.txt
msi@msi-debian:~$ chmod go-w test.txt
msi@msi-debian:~$ ls -l test.txt
-rwxr-xr-x 1 msi msi 5 8月30日 21:00 test.txt
msi@msi-debian:~$
```

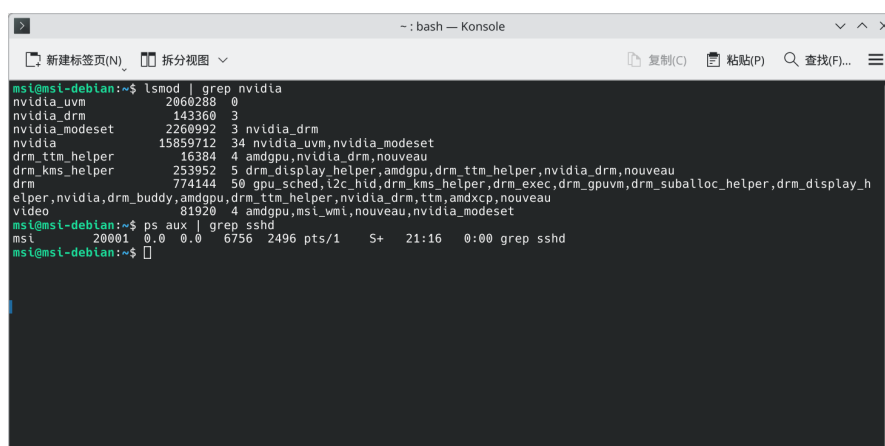
2.1.5 管道与 grep 配合实现快速查找

利用管道将输出较长的命令交给 grep 来查找，能够大幅提高查找效率。

```
# 查找 Nvidia 驱动
lsmod | grep nvidia

# 检查 SSH 进程是否运行
ps aux | grep sshd
```

在查找驱动、服务或进程时，grep 可显著缩短查找时间。运行结果如下：



```
msi@msi-debian:~$ lsmod | grep nvidia
nvidia_uvm      2060288  0
nvidia_drm      143360  3
nvidia_modeset  2260992  3 nvidia_drm
nvidia          15859712 34 nvidia_uvm,nvidia_modeset
drm_ttm_helper   16384  4 amdgpu,nvidia_drm,nouveau
drm_kms_helper   253952  5 drm_display_helper,amdgpu,drm_ttm_helper,nvidia_drm,nouveau
drm              774144  50 gpu_sched,t2e_hid,drm_kms_helper,drm_exec,drm_gpuvm,drm_suballoc_helper,drm_display_h
elper,nvidia,drm_buddy,amdgpu,drm_ttm_helper,nvidia_drm,ttm,amdxcp,nouveau
video            81920  4 amdgpu,msi_wmi,nouveau,nvidia_modeset
msi@msi-debian:~$ ps aux | grep sshd
msi      20001  0.0  0.0  6756  2496 pts/1    S+   21:16   0:00 grep sshd
msi@msi-debian:~$
```

2.2 Git 相关

2.2.1 使用 SSH 密钥简化推送步骤

使用 SSH 密钥可以避免每次推送时都输入用户名和密码，更安全便捷。

以 Github 为例：

1. 生成 SSH 密钥

使用 `ssh-keygen` 命令生成一个 SSH 密钥（此处以 RSA 算法为例）。

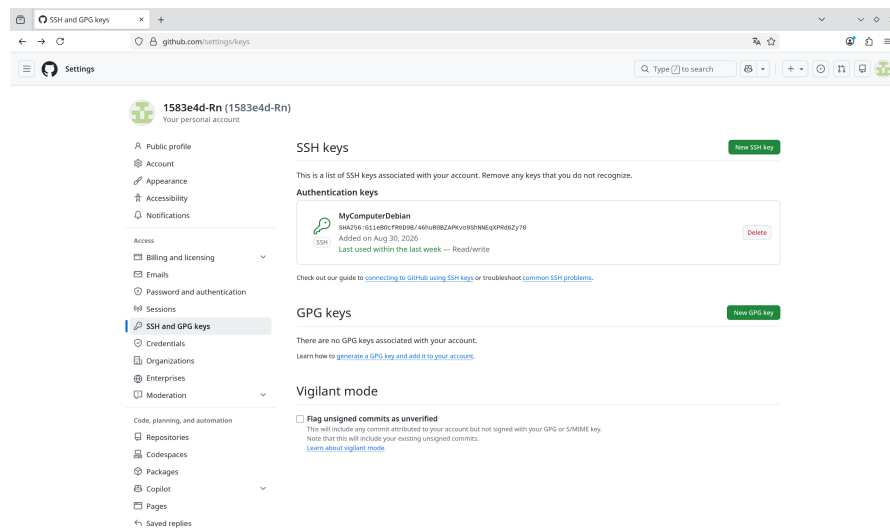
```
ssh-keygen -t RSA -C "name@example.com"
```

生成的密钥默认会存储在 `~/.ssh` 目录下。

2. 绑定 SSH 密钥

将生成的 SSH 密钥公钥绑定至 Github 账户（注意不可泄漏私钥）。

打开 `~/.ssh/id_rsa.pub`，复制其中的内容。登录 Github 并打开账户设置，选择“SSH and GPG keys”，单击“New SSH key”，输入名称并粘贴公钥，保存即可。



这样，只需在关联远程仓库时，使用 SSH 链接，便可在推送时自动验证身份。

2.2.2 向远程仓库推送代码

仍以 Github 为例：

1. 克隆远程仓库

```
# 使用 SSH 协议克隆仓库（需完成 SSH 密钥配置）
git clone git@github.com:username/demo-repo.git
```

2. 关联远程仓库

将克隆的仓库与远程仓库关联。

```
# 使用 SSH 协议（习惯上将远程仓库命名为 origin）
git remote add origin git@github.com:username/demo-repo.git
```

3. 进行本地操作

与本地仓库一样，可以进行签出分支、上传文件、提交与合并等操作。

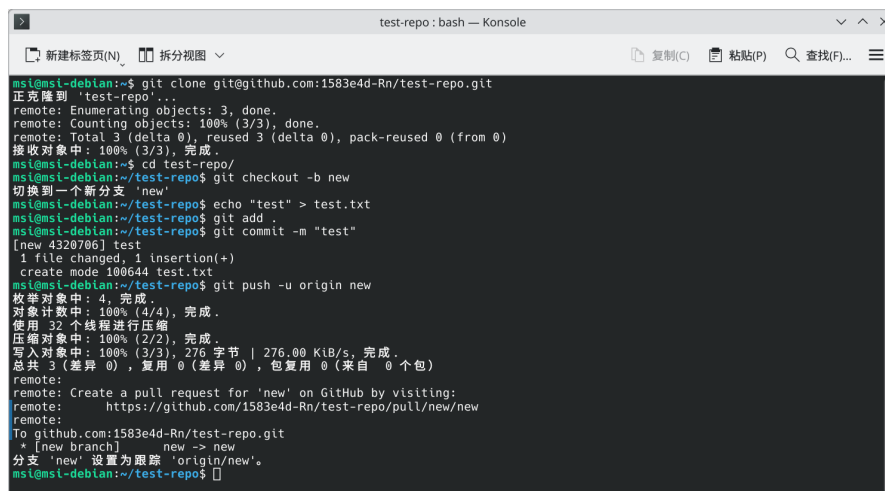
4. 进行远程推送

本地操作完成后，将修改的文件推送到远程仓库。

```
git push origin
```

若签出了新分支，或远程分支的名字和本地不一样，需加上参数-u(-set-upstream) 来设置上游跟踪（建立本地分支与远程分支间的关联）。

示例如下：



```
msi@msi-debian:~$ git clone git@github.com:1583e4d-Rn/test-repo.git
正克隆到 'test-repo'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
接收对象中: 100% (3/3), 完成.
msi@msi-debian:~$ cd test-repo/
msi@msi-debian:~/test-repo$ git checkout -b new
切换到一个新分支 'new'
msi@msi-debian:~/test-repo$ echo "test" > test.txt
msi@msi-debian:~/test-repo$ git add .
msi@msi-debian:~/test-repo$ git commit -m "test"
[new 4320706] test
1 file changed, 1 insertion(+)
create mode 100644 test.txt
msi@msi-debian:~/test-repo$ git push -u origin new
枚举对象中: 4, 完成.
对象计数中: 100% (4/4), 完成.
使用 32 个线程进行压缩
压缩对象中: 100% (2/2), 完成.
写入对象中: 100% (3/3), 276 字节 | 276.00 KiB/s, 完成.
总共: 3 (差异 0), 复用 0 (差异 0), 包复用 0 (来自 0 个包)
remote: Create a pull request for 'new' on GitHub by visiting:
remote:   https://github.com/1583e4d-Rn/test-repo/pull/new/new
remote:
To github.com:1583e4d-Rn/test-repo.git
 * [new branch]      new -> new
分支 'new' 设置为跟踪 'origin/new'.
```

2.3 Latex

2.3.1 文档结构控制

使用特定的宏包，可实现文档结构控制。

```
% 设置页眉页脚
\usepackage{fancyhdr}
\pagestyle{fancy}
\fancyhf{}
\fancyhead[L]{\small 示例文本 1}
\fancyhead[R]{\small 示例文本 2}
\fancyfoot[C]{\thepage}

% 设置页边距与纸张大小
\usepackage[a4paper, margin=2.5cm]{geometry}

% 目录生成（一般放在标题页或摘要后，正文开始前）
```

```
\tableofcontents % 在这里生成目录  
\newpage % 目录独占一页，正文从下一页开始
```

具体效果参考本报告。

2.3.2 创建无序列表与有序列表

在文档中创建列表，使文章层次结构清晰。

```
\textbf{无序列表：}  
\begin{itemize}  
  \item 第一个项目  
  \item 第二个项目  
\end{itemize}  
  
\textbf{有序列表：}  
\begin{enumerate}  
  \item 第一步  
  \item 第二步  
\end{enumerate}
```

示例效果： **无序列表：**

- 第一个项目
- 第二个项目

有序列表：

1. 第一步
2. 第二步

2.3.3 插入图片

通常将图片放入浮动体 figure 中，以自动适配位置。

```
\documentclass{article}
\usepackage{graphicx}
\graphicspath{{figures/}} % 假设图片在 figures 文件夹下

\begin{document}
\begin{figure}[H]
  \centering
  \includegraphics[width=0.8\textwidth]{example-image} % 图片文件名
  \caption{这是一个示例图片}
  \label{fig:example}
\end{figure}
如图 \ref{fig:example} 所示。
\end{document}
```

效果如下：

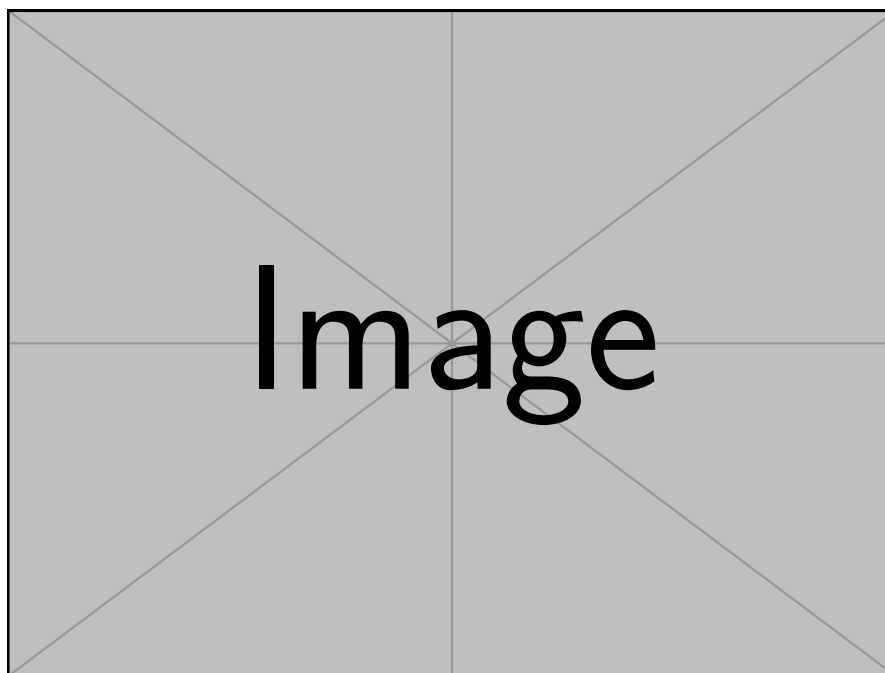


图 3: 这是一个示例图片

3 感悟

通过第一周的实验练习，形成以下认识和体会：

- Shell 管道组合的有效性：利用 `awk`、`sort`、`uniq`、`head` 等工具构建管道，仅需数行命令即可完成日志文件的统计与分析。
- Shell 脚本的健壮性设计：在编写 `analyze.sh` 的过程中，除实现核心功能外，还需对参数数量、文件存在性等边界条件进行检查。
- Git 分支合并冲突的处理：冲突产生的根本原因在于不同分支对同一文件的同一区域进行了修改。解决冲突的关键在于理解各分支修改的意图，并据此做出合理选择。

- LaTeX 交叉引用的编译要求：初次编译 LaTeX 文档时，交叉引用显示为?? 是因为引用信息需要在多次编译后才能正确解析。采用 latexmk 工具可自动化该流程。

Shell 负责文件操作与文本处理，Git 实施版本管理，LaTeX 完成报告排版。三者协同使用所形成的工具链，其整体效率增益显著超过各工具功能的简单叠加，体现了系统性工程方法的价值。

Github 提交

此处上传 Github 提交记录。(报告于完成后上传)

